

Project Milestone 2 - Report

Swann Ade, Antoine Heurtier and Elyes Ben Abid
CS-233 - Introduction to Machine Learning

1 Introduction

This report presents our implementation of two machine learning methods applied to the Gaming and Mental Health Dataset, a dataset composed of 2000 samples with 13 features. The goal is to predict either the addiction score (regression, 0-10) or the addiction level (classification, Low/Medium/High) from gaming habits.

We implement a Multi-Layer Perceptron (MLP) and K-Means clustering from scratch using course formulas through NumPy, including a hand-written forward pass, back-propagation and weight updates for the MLP. We evaluate them using MSE for regression and accuracy/F1-score for classification.

2 Data Preparation

The 1600 provided training samples were split 80/20 into 1280 training and 320 validation samples using a shuffled permutation. The validation set was used to tune hyperparameters without touching the test set, and this is done by passing the `--test` flag.

Feature normalization is then applied after the split: means and standard deviations are computed on the training set only, then applied to both training and validation sets, as computing statistics before the split would "leak" validation information into the model. A bias term is appended to the feature matrix for the MLP, but skipped for K-Means since a constant column carries no information for this Euclidian distance-based method.

3 Methods

3.1 Multi-Layer Perceptron (MLP)

We implemented a flexible Multi-Layer Perceptron capable of handling both multi-class classification and continuous regression tasks. The network architecture dynamically adjusts its dimensions based on the target task: the input layer dimension is fixed at $D = 14$ (13 normalized features plus a constant bias term), while the output layer structure transitions between $C = 3$ neurons (one-hot encoded targets) for classification and a single neuron ($C = 1$) for regression.

3.1.1 Mathematical Framework

For a network with L layers, the forward pass for a given batch input $\mathbf{a}^{(0)} = \mathbf{X}$ computes the pre-activations $\mathbf{z}^{(l)}$ and activations $\mathbf{a}^{(l)}$ for each layer $l \in \{1, \dots, L\}$ recursively:

$$\mathbf{z}^{(l)} = \mathbf{a}^{(l-1)}\mathbf{W}^{(l)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = f^{(l)}(\mathbf{z}^{(l)})$$

where $\mathbf{W}^{(l)}$ and $\mathbf{b}^{(l)}$ represent the weight matrices and bias vectors respectively, and $f^{(l)}$ denotes the element-wise activation function (*ReLU* or σ). During the backward pass, the error delta $\delta^{(L)}$ at the output layer is initiated using the gradient of the loss function (Mean Squared Error for regression and for classification) combined with the output activation derivative:

$$\delta^{(L)} = \nabla_{\mathbf{a}^{(L)}} \mathcal{L} \odot f^{(L)'}(\mathbf{z}^{(L)})$$

This error is backpropagated through the hidden layers using the chain rule, and the gradients with respect to the parameters are accumulated over the mini-batch of size N :

$$\delta^{(l)} = \left(\delta^{(l+1)}(\mathbf{W}^{(l+1)})^T \right) \odot f^{(l)'}(\mathbf{z}^{(l)}), \quad \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}} = \frac{1}{N}(\mathbf{a}^{(l-1)})^T \delta^{(l)}, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}} = \frac{1}{N} \sum_{i=1}^N \delta_{i, \cdot}^{(l)}$$

3.1.2 Training Implementation

Weights are initialized using a normal distribution ($\mathbf{W} \sim \mathcal{N}(0, \frac{1}{n_{in}})$) to prevent gradient issues. Optimization is conducted via Mini-Batch Gradient Descent. At each epoch, the training set points are shuffled using a random permutation to break sample ordering dependencies before partitioning into mini-batches. Parameters are updated using a fixed learning rate η :

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(l)}}, \quad \mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \frac{\partial \mathcal{L}}{\partial \mathbf{b}^{(l)}}$$

3.2 K-Means

As seen in class, K-Means is an unsupervised clustering algorithm that we adapt to classification through "majority voting". The procedure initializes K cluster centers $\{\mu_k\}_{k=1}^K$ at random data points, then alternates between two steps until convergence:

1. Assign each point $\mathbf{x}_i$ to its closest center using the Euclidean distance:

$$c_i = \arg \min_k \|\mathbf{x}_i - \mu_k\|_2$$

2. Update each center as the mean of its assigned points:

$$\mu_k = \frac{1}{|C_k|} \sum_{i \in C_k} \mathbf{x}_i$$

Going from `n_init=1` to `n_init=10` improved the validation F1 from 0.27 to 0.43 with $K = 3$. This is a substantial gain and this choice of $K = 3$ is also the most natural one for our task, since the classification target itself has exactly three levels.

The number of clusters K is the main hyperparameter. We performed a sweep over $K \in \{1, \dots, 17\}$ with `n_init=50` to avoid initialization noise and plotted both validation accuracy and macro F1 against K . The results clearly show that $K = 3$ gives the best trade-off between both metrics, which is consistent with the fact that the target classification problem itself contains three addiction levels (Low/Medium/High).

Convergence is reached when centers stop moving between iterations. For classification, each cluster is then labeled by majority vote over the training labels of its assigned points, and a test sample is predicted by using the label of its closest cluster.

Since K-Means is highly sensitive to the random initialization of its centers we mitigated this by implementing an `--n_init` parameter: the algorithm is restarted multiple times and keeps the run with the highest training accuracy.

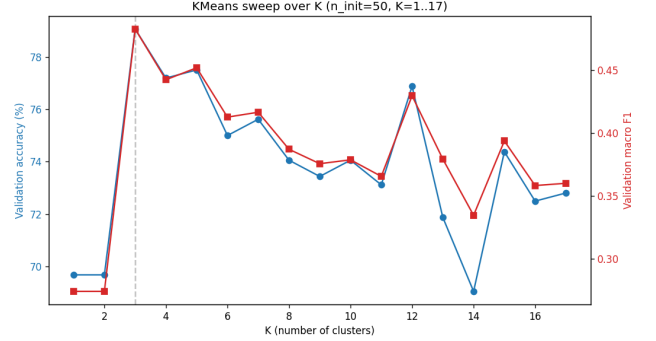


Figure 1: Validation accuracy and macro F1 as a function of K (with `n_init=50`). Best results at $K = 3$.

4 Results

Method	F1	MSE	Accuracy	Train time	Predict time
MLP (Regression)	-	1.0288	-	469.12 ms	0.02 ms
MLP (Classification)	0.732	-	84.5%	1588 ms	0.35 ms
K-Means ($K = 3$, <code>n_init=8</code>)	0.47	-	79.75%	1144.96 ms	1.20 ms

Table 1: Test set results and runtime (*tested on an Apple Macbook Pro M3*).

5 Discussion & Conclusion

Performance: The MLP outperformed K-Means on the classification task, achieving a higher accuracy (84.5% vs 79.75%) and a better macro F1-score (0.732 vs 0.47). This is expected since MLP is a supervised method that directly optimizes a classification loss, while K-Means only finds geometric clusters and is then forced to classify via majority voting, in brief, it’s unsupervised. On the regression task, the MLP reached an MSE of 1.0288, which suggests that the relationship between gaming habits and addiction score is mostly linear and could be captured by a small neural network.

Hyperparameters and Visualization: For K-Means, the sweep over K clearly identifies $K = 3$ as the optimum, and the `n_init` restart strategy roughly doubled the macro F1 by escaping bad random initializations. For the MLP, we tuned the learning rate η , the number of epochs, the batch size and the hidden layer sizes by simply monitoring the training and validation loss curves. This allowed us to detect overfitting and to pick good learning rate. We observed that very deep/wide architectures did not bring noticeable gains, likely because the 13 input features are too few to justify a

fat model. We added in the `"/tuning"` directory scripts that allowed us to find the best hyper params : `tune_mlp.py` and `tune_kmeans.py`.

Experimental Explorations: For the MLP, we experimented with different activation functions (ReLU vs sigmoid) and tested several hidden layer configurations to compare their impact on both tasks. We also explored applying the same MLP backbone to regression and classification by simply swapping the loss function and output layer, which confirmed that this generalizes well across both formulations of the problem.

Efficiency: K-Means is a bit faster to train than the MLP (around 1144 ms vs 1588 ms), despite the fact that we run it 8 times to get the best result. At each step it only performs a few iterations of distance computations and mean updates, whereas the MLP requires hundreds of forward and backward passes over the entire training set. Prediction time is comparable for both methods (around 1 ms) and remains negligible. Thus said, the MLP is worth its extra training cost when the goal is to maximize predictive performance!

In conclusion, both methods successfully extract a relationship between gaming habits and addiction, but the supervised MLP delivers clearly stronger predictions on both regression and classification at the cost of more computations/time, confirming that gaming behavior contains meaningful information about a player’s mental health.